



Lab 9

Sequential Logic – Part 2

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Contents of this slide are provided by NTOU VLSI Lab601

Objectives

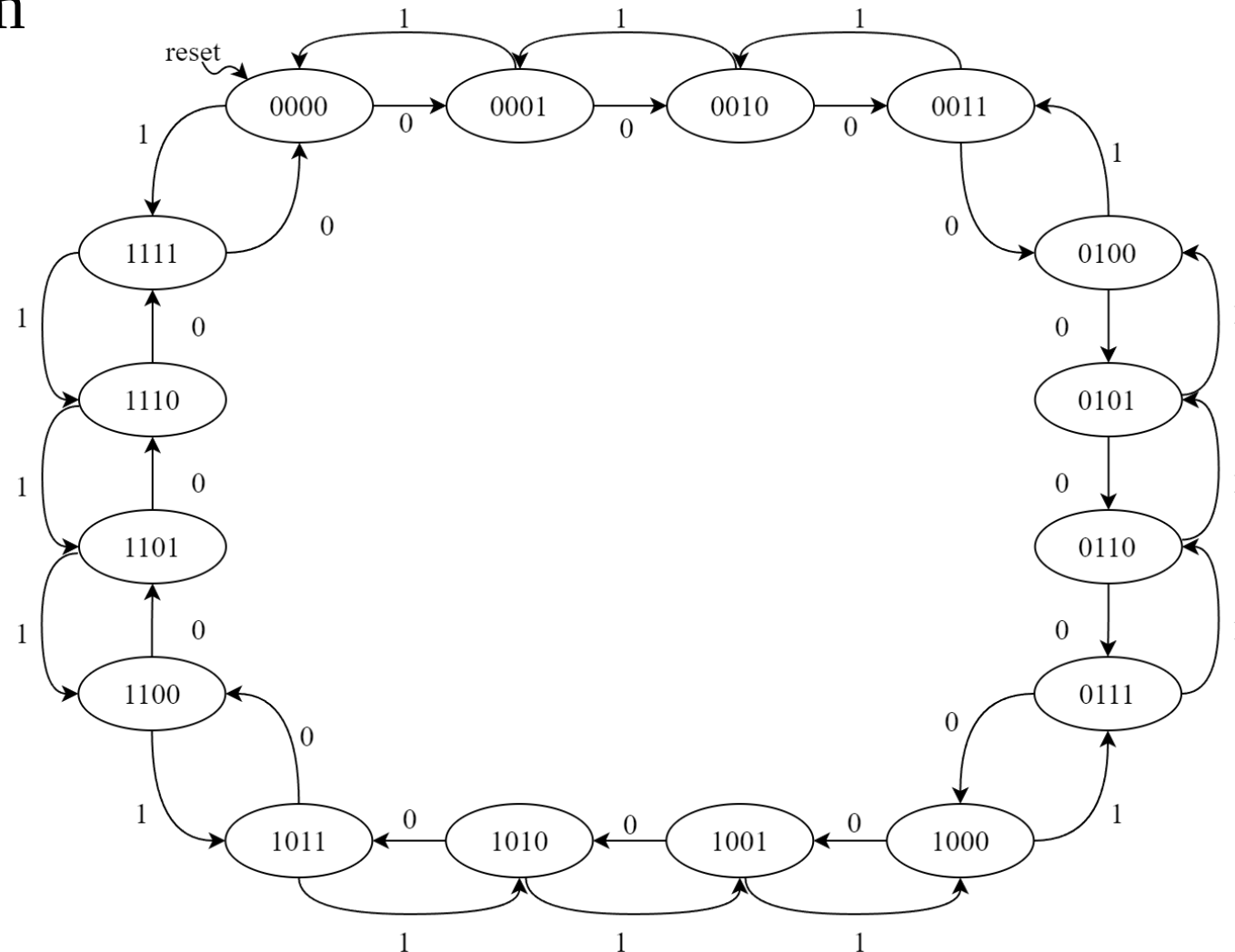
- Counter
- Traffic Light

LAB 1 – 4-bit Synchronous Up/Down Counter (1/8)

- Use four *D* flip-flops to design a 4-bit synchronous up/down counter.
 - When $x = 0$, the counter counts up.
 - When $x = 1$, the counter counts down.
- Performed signal simulation and downloaded to DE0 for verification.
- Signal settings:
 - Input: *clk*, *reset*, and x
 - Output: $q[3:0] = \{A, B, C, D\}$

LAB 1 – 4-bit Synchronous Up/Down Counter (2/8)

- State diagram



LAB 1 – 4-bit Synchronous Up/Down Counter (3/8)

- State table

Present State (t)					Next State ($t + 1$)			
x	A	B	C	D	A	B	C	D
0	0	0	0	0	0	0	0	1
0	0	0	0	1	0	0	1	0
0	0	0	1	0	0	0	1	1
0	0	0	1	1	0	1	0	0
0	0	1	0	0	0	1	0	1
0	0	1	0	1	0	1	1	0
0	0	1	1	0	0	1	1	1
0	0	1	1	1	1	0	0	0
0	1	0	0	0	1	0	0	1
0	1	0	0	1	1	0	1	0
0	1	0	1	0	1	0	1	1
0	1	0	1	1	1	1	0	0
0	1	1	0	0	1	1	0	1
0	1	1	0	1	1	1	1	0
0	1	1	1	0	1	1	1	1
0	1	1	1	1	0	0	0	0

Present State (t)					Next State ($t + 1$)			
x	A	B	C	D	A	B	C	D
1	0	0	0	0	1	1	1	1
1	0	0	0	1	0	0	0	0
1	0	0	1	0	0	0	0	1
1	0	0	1	1	0	0	1	0
1	0	1	0	0	0	0	1	1
1	0	1	0	1	0	1	0	0
1	0	1	1	0	0	1	0	1
1	0	1	1	1	0	1	1	0
1	1	0	0	0	0	1	1	1
1	1	0	0	1	1	0	0	0
1	1	0	1	0	1	0	0	1
1	1	0	1	1	1	0	1	0
1	1	1	0	0	1	0	1	1
1	1	1	0	1	1	1	0	0
1	1	1	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

LAB 1 – 4-bit Synchronous Up/Down Counter (4/8)

- Testbench

```

module testbench;

    logic clk, reset, x;
    logic [3:0] q;

    up_down_counter_4bits counter_4bits_0(
        .clk(clk),
        .reset(reset),
        .x(x),
        .q(q)
    );

    always #5 clk = ~clk;

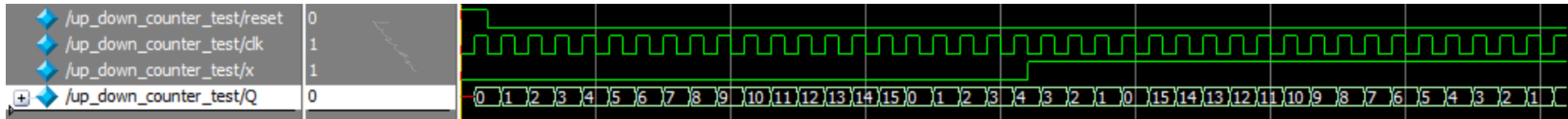
    initial begin
        clk = 0; reset = 1; x = 0;
        #10 reset = 0;
        #200 x = 1;
        #200 $stop;
    end

endmodule

```

LAB 1 – 4-bit Synchronous Up/Down Counter (5/8)

- Waveform

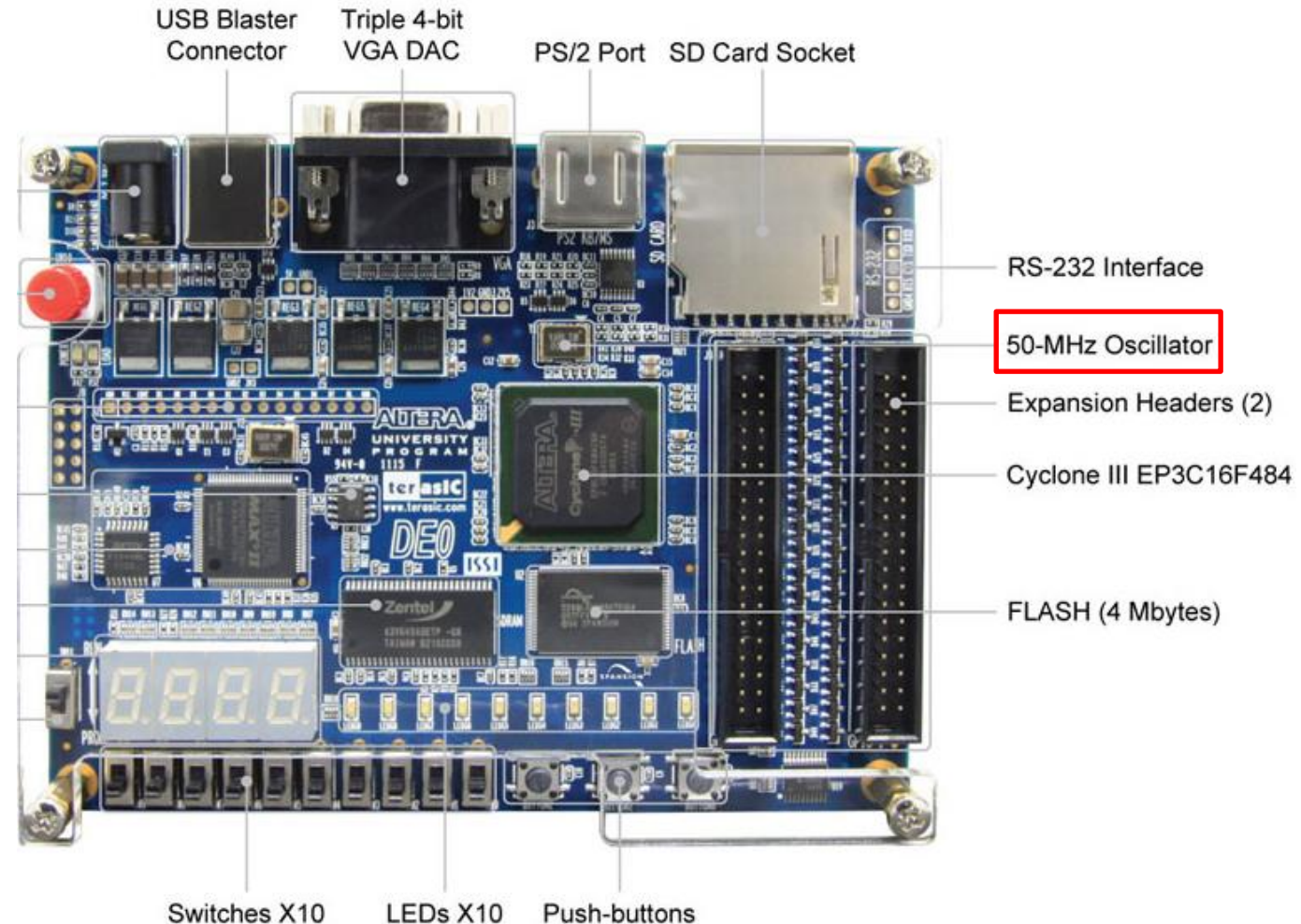


- DE0 pin settings

Signals	Pins
<i>clk</i>	CLK
<i>reset</i>	Button (~BTN[1])
<i>x</i>	Switch (SW[0])
<i>q</i>	SSD (HEX0)

LAB 1 – 4-bit Synchronous Up/Down Counter (6/8)

- The clock will be provided by the internal 50 MHz CLK of the FPGA, i.e., DE0.
- The changes of the digits of the SSD are difficult to be observed if 50 MHz is used as the clock of the design counter.
- A frequency divider is needed to reduce the clock frequency for easier observation.



Lab 1 – 4-bit Synchronous Up/Down Counter (7/8)

Frequency Divider



```
module freg_div_23(  
    input clk,  
    input reset,  
    output logic clk_out  
);  
  
    logic [31:0] q;  
  
    always @(posedge clk or posedge reset)  
    begin  
        if(reset)  
            q <= 0;  
        else  
            q <= q + 1;  
        end  
  
        assign clk_out = q[23];  
        // Reducing the number after 'q' will increase the clock frequency.  
    endmodule
```

- Question: By what value is 50 MHz divided by?

Lab 1 – 4-bit Synchronous Up/Down Counter (8/8)

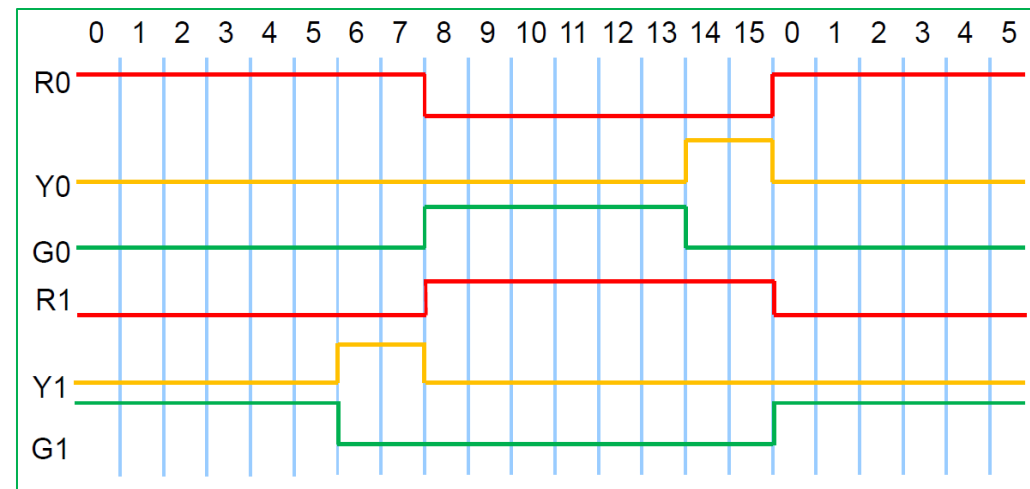
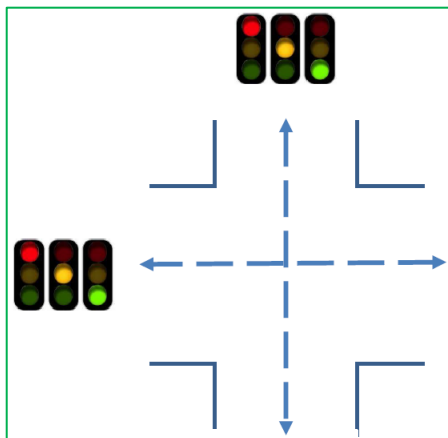
Partial Codes



```
freg_div_23 u_freg_div_23(  
    .clk(CLK),  
    .reset(~BTN[1]),  
    .clk_out(clk)  
);  
  
up_down_counter_4bits counter_4bits_0(  
    .clk(clk),  
    .reset(~BTN[1]),  
    .x(SW[0]),  
    .q(q)  
);  
  
seven_segment_decoder u_seven_segment_decoder(  
    .a(q),  
    .seg(HEX0)  
);
```

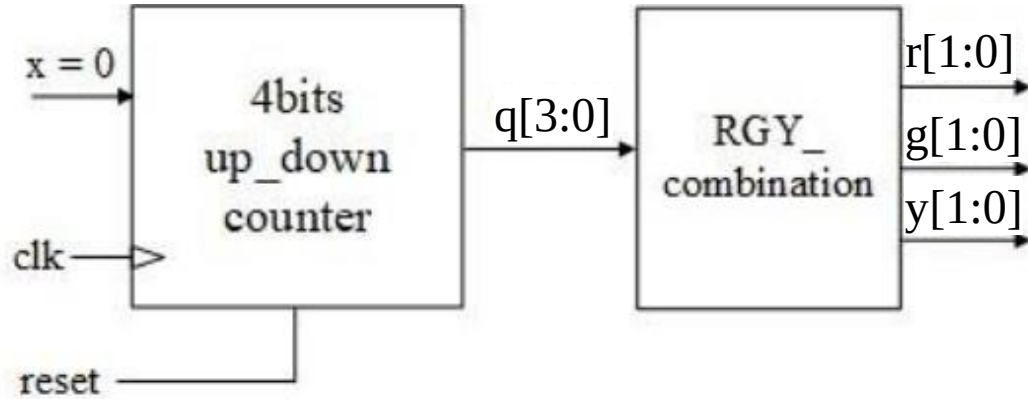
LAB 2 – Traffic Light (1/4)

- Display sequences of the traffic light set #0 are $r \rightarrow g \rightarrow y \rightarrow r \rightarrow \dots$
 $(r[0], g[0], y[0]) = (1, 0, 0) \xrightarrow{8CC} (0, 1, 0) \xrightarrow{6CC} (0, 0, 1) \xrightarrow{2CC} (1, 0, 0)$
- Display sequences of the traffic light set #1 are $g \rightarrow y \rightarrow r \rightarrow g \rightarrow \dots$
 $(r[1], g[1], y[1]) = (0, 1, 0) \xrightarrow{6CC} (0, 0, 1) \xrightarrow{2CC} (1, 0, 0) \xrightarrow{8CC} (0, 1, 0)$
 - ‘r’ represents red light, ‘g’ represents green light, ‘y’ represents yellow light;
 - $r[0]=1$ means the red light of the traffic light set #0 is on, $r[0]=0$ means it is off.
 - CC means the clock cycle.



LAB 2 – Traffic Light (2/4)

● Architecture diagram & State



● Signal settings

- Input: x , clk , and $reset$
- Output: $r[1:0]$, $g[1:0]$, $y[1:0]$

● State table

Present State				Output					
Q_3	Q_2	Q_1	Q_0	R_0	G_0	Y_0	R_1	G_1	Y_1
0	0	0	0	1	0	0	0	1	0
0	0	0	1	1	0	0	0	1	0
0	0	1	0	1	0	0	0	1	0
0	0	1	1	1	0	0	0	1	0
0	1	0	0	1	0	0	0	1	0
0	1	0	1	1	0	0	0	1	0
0	1	1	0	1	0	0	0	0	1
0	1	1	1	1	0	0	0	0	1
1	0	0	0	0	1	0	1	0	0
1	0	0	1	0	1	0	1	0	0
1	0	1	0	0	1	0	1	0	0
1	0	1	1	0	1	0	1	0	0
1	1	0	0	0	1	0	1	0	0
1	1	0	1	0	1	0	1	0	0
1	1	1	0	0	0	1	1	0	0
1	1	1	1	0	0	1	1	0	0

LAB 2 – Traffic Light (3/4)

• Sample Code

```

module RGY_top(
    input clk,
    input reset,
    output [1:0] r, g, y
);
    logic [3:0] q;

    up_down_counter_4bits c1(
        .clk(clk),
        .reset(reset),
        .x(1'b0),
        .q(q)
    );

    RGY_combination RGY(
        .q(q),
        .r(r),
        .g(g),
        .y(y)
    );
endmodule

```

• Testbench

```

module testbench;
    logic clk, reset;
    logic [1:0] r, y, g;

    RGY_top RGY(
        .clk      (clk      ),
        .reset    (reset    ),
        .r        (r        ),
        .y        (y        ),
        .g        (g        )
    );

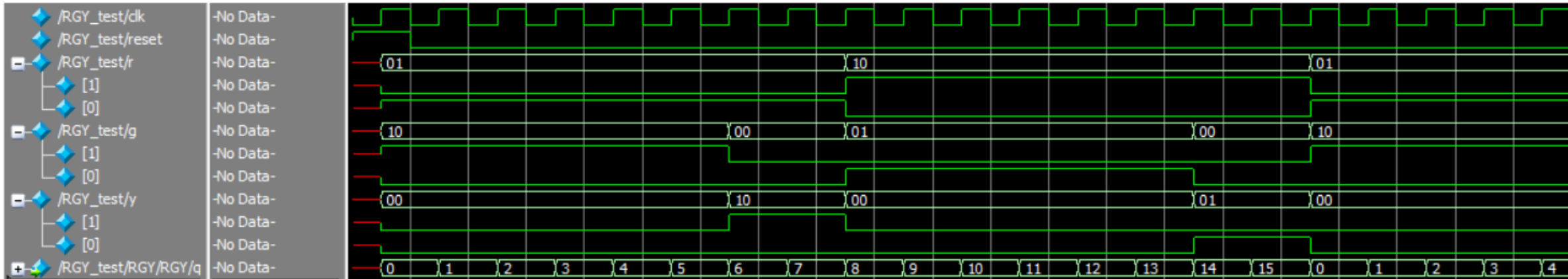
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1;
        #10 reset = 0;
        #200 $stop;
    end
endmodule

```

LAB 2 – Traffic Light (4/4)

● Waveform



● DE0 pin settings

Signals	Pins
<i>clk</i>	CLK
<i>reset</i>	~BTN[1]
r[1], r[0]	LED[7], LED[2]
y[1], y[0]	LED[6], LED[1]
g[1], g[0]	LED[5], LED[0]